

EXPERIMENT - 1

Linux Programming Environment & Basic C Programming

Aim: To set up a Linux programming environment and create, compile, and execute a simple C program using the GCC compiler.

A) Setting up the Linux Programming Environment

C programming can be practiced using either an online or offline environment.

Online Method

An online Linux environment such as CoCalc can be used to write, compile and execute C programs without installing Linux in windows.

1. Open the online Linux environment and sign in.
2. Create or open a Linux project/terminal.
3. Create a C source file with the .c extension.
4. Compile the program using GCC.
5. Run the generated executable and observe the output.

Offline Methods

Dual boot: Dual boot means installing two operating systems on the same computer.

Single boot: Linux is installed as the only operating system on the computer.

Bootable pen drive: A bootable USB contains an operating system that the computer can boot from.

Virtual machine (VM): A virtual machine is a software based computer that can run an operating system inside another computing environment.

Virtual Machine Manager (Hypervisor)

Type-1 (Bare-metal): Runs directly on the physical hardware without a host operating system. Examples include VMware ESXi and Microsoft Hyper-V Server.

Type-2 (Hosted): Runs as an application on a host operating system. Examples include Oracle VirtualBox and VMware Workstation. Oracle VirtualBox can be used for this experiment.

Installation of VirtualBox and Kali Linux VM

6. Download Oracle VirtualBox for the host operating system.
7. Download a pre-built Kali Linux virtual machine for VirtualBox.
8. Extract the downloaded Kali Linux VM file.
9. Install Oracle VirtualBox.
10. Open VirtualBox.
11. Open/import the saved Kali Linux VM.
12. Start the virtual machine and log in to Kali Linux.
13. Open the terminal to begin C programming.

Download VirtualBox

The VirtualBox Extension Pack is available for personal and educational use on this page under the PUEL license. The VirtualBox Extension Pack is also available under commercial or enterprise terms. By downloading, you agree to the terms and conditions of the respective license.

VirtualBox Platform Packages

VirtualBox 7.2.14 platform packages

- Windows hosts**
- macOS / Intel hosts**
- Linux distributions**
- Solaris hosts**
- Solaris 11 IPS hosts**

Platform packages are released under the terms of the GPL version 3

VirtualBox Extension Pack VirtualBox 7.2.14 Extension Pack

This VirtualBox Extension Pack Personal Use and Educational License governs your access to and use of the VirtualBox Extension Pack. It does not apply to the VirtualBox base package and/or its source code, which are licensed under version 3 of the GNU General Public License "GPL".

See our [FAQ](#) for answers to common questions.

VirtualBox Extension Pack Personal Use and Educational License (PUEL)

[PUEL License FAQ](#)[PUEL License Text](#)[Accept and download](#)

Change Log [List of changes](#)

File Checksums [SHA256 checksums](#) [MD5 checksums](#)

User Guide [User Guide \(PDF\)](#) [User Guide \(HTML\)](#)

<https://download.virtualbox.org/virtualbox/7.2.14/VirtualBox-7.2.14-174565-Win.exe>

Choose your Kali |

Linux ☒ Docker

Installer Images

- ✓ Direct access to hardware
- ✓ Convenient disk format
- ✓ No overhead

Single or multiple bootable Kali, allows you complete control over the hardware access (perfect for in-built WiFi and GPUs), enabling the best performance.

[Recommended](#)

Virtual Machines

- ✓ Complete functionality
- ✓ Complete encapsulation
- ✓ Customizable file format
- ✓ Limited guest access to hardware
- ✓ Higher system requirements

VirtualBox & VirtualBox-pro built images. Allowing for a Kali install without affecting the host OS with additional features such as snapshots, logon images for quick spin-up after shutdown.

[Recommended](#)

ARM

- ✓ Range of hardware from the latest Cortex devices (up to highest-end servers)
- ✗ System architecture limits various packages
- ✗ Not always containerized kernel

Works on relatively inexpensive & low-powered Single Board Computers (SBCs) as well as modern ARM-based laptops, which combine high speed with long battery life.

Mobile

- ✓ Kali installed on Android
- ✓ Kali in your pocket, on the go
- ✓ Access to native payment apps

A mobile generation using platform for Android devices, based on Kali Linux. Kali host-kernel consists of an Android App, App Store, Kali Container, and Kali.

Cloud

- ✓ Cloud deployments
- ✓ Can leverage provider's resources
- ✗ Provider may become costly
- ✗ Not always containerized kernel

Hosting providers which have Kali Linux pre-installed, ready to go, without worrying about infrastructure maintenance.

Containers

- ✓ Low overhead to access full kernel
- ✗ Userland actions only
- ✗ Not full containerized kernel
- ✗ No direct access to hardware

Using Docker or LXD, allows for extremely quick and easy access to Kali's tool set without the overhead of an isolated virtual machine.

Live Boot

- ✓ Lightweight host system
- ✓ Direct access to hardware
- ✓ Containerized Kali kernel
- ✗ Performance decreases when heavy I/O

Quick and easy access to a full Kali install. Your Kali, always with you, without altering the host OS, plus allows you to benefit from hardware access.

WSL

- ✓ Access to the full kernel through the WSL framework
- ✗ Userland actions only
- ✗ Not full containerized kernel
- ✗ No direct access to hardware

Windows Subsystem for Linux (WSL) is included out of the box with modern Windows. Use Kali (and Windows) without installing additional software.

[Build Scripts](#)

<https://www.kali.org/get-kali/#kali-virtual-machines>

Oracle VirtualBox 7.2.14 Installer

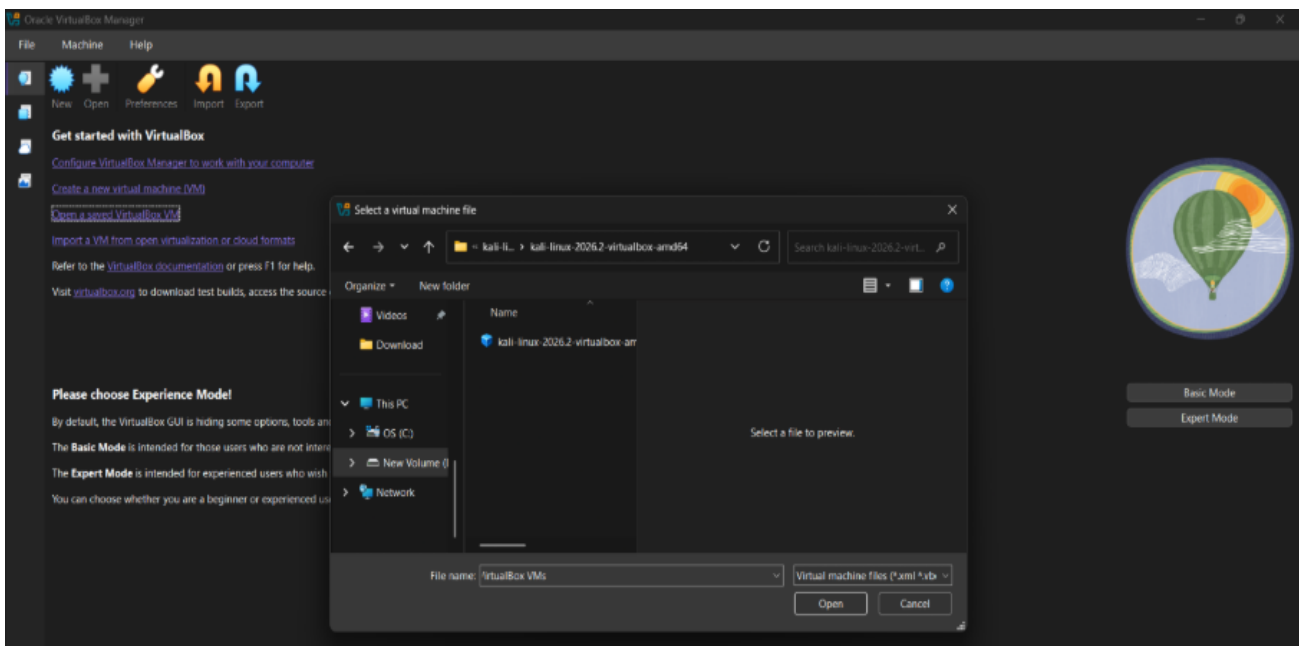
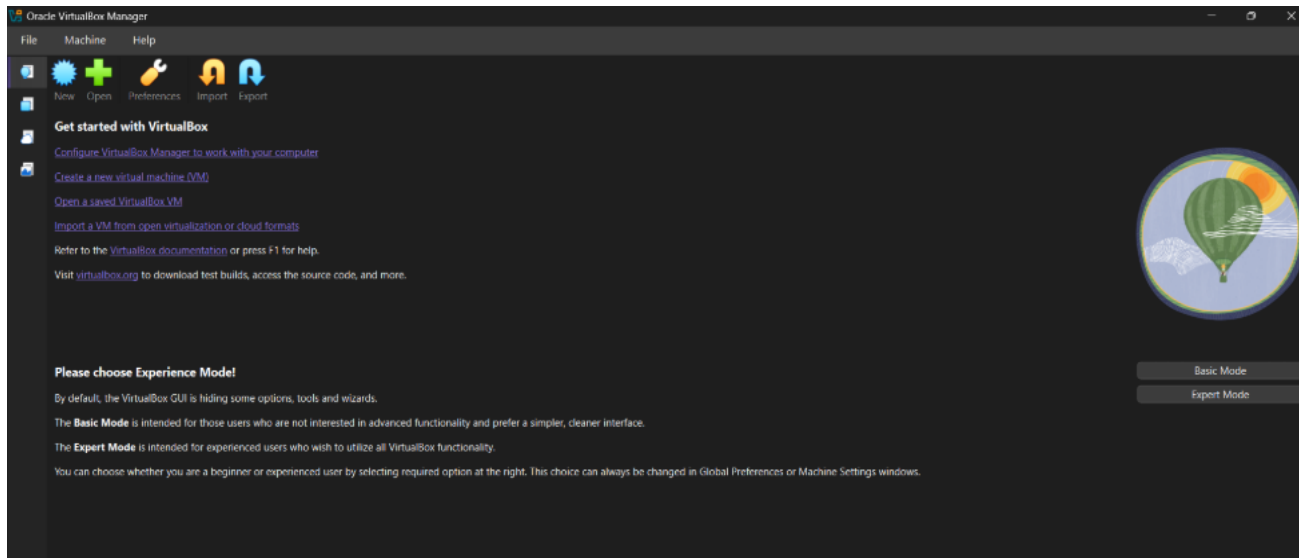
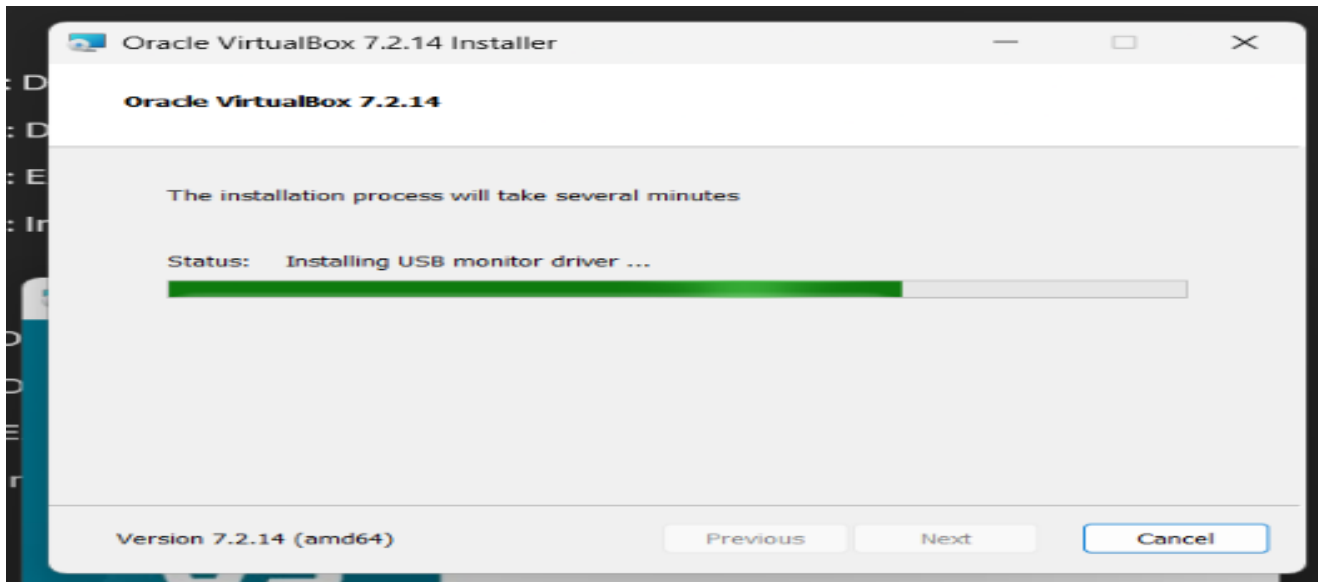


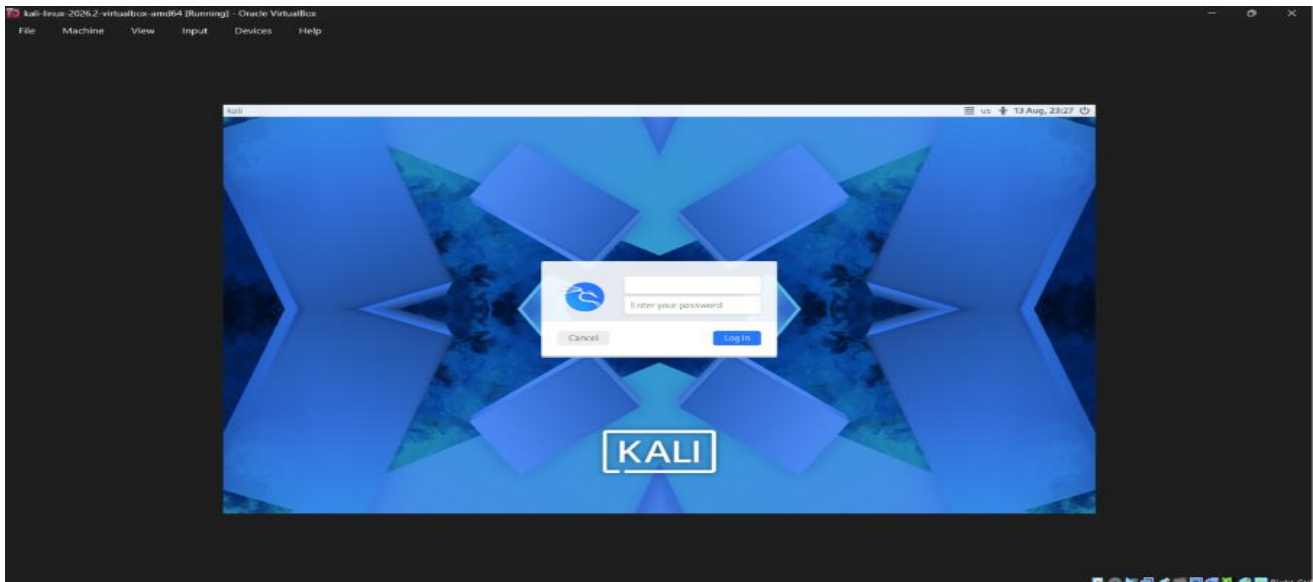
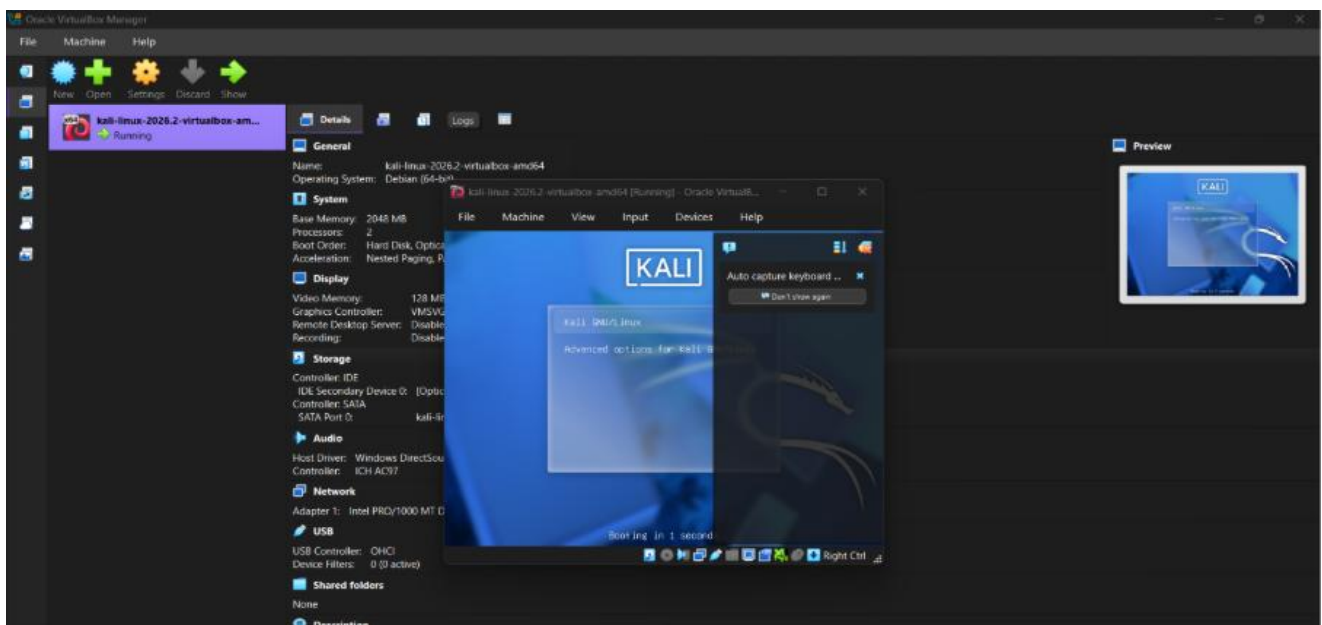
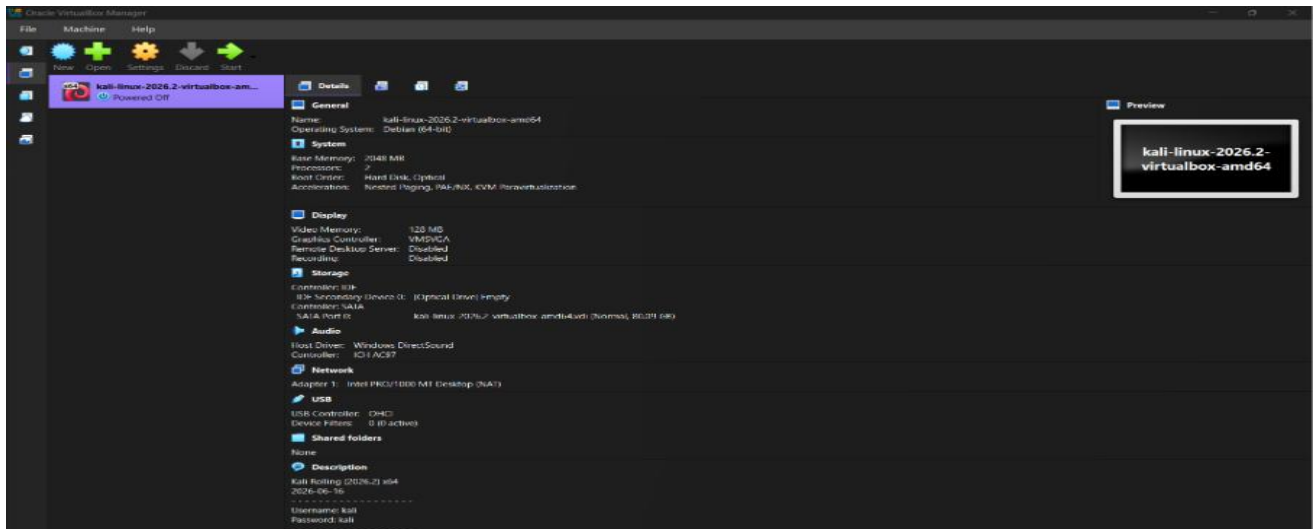
Welcome to the Oracle VirtualBox 7.2.14 Installer

The Installer will install Oracle VirtualBox 7.2.14 on your computer. Click Next to continue or Cancel to exit the Installer.

Version 7.2.14 (amd64)

[Next](#)[Cancel](#)





Thus, the installation of VMM(Hypervisor). Type 2 Oracle (Virtual Box) & VM (Kali Linux) is completed.

B) Write, Compilation and Execution of a Simple C Program

Open the terminal and create a simple C program to understand the working of the Linux programming environment.

Program:

```
#include <stdio.h>

int main(void)
{
    printf("My Name Is Aryan Mehendiratta\n");

    return 0;
}
```

The program is saved in a file named `exp1.c`.

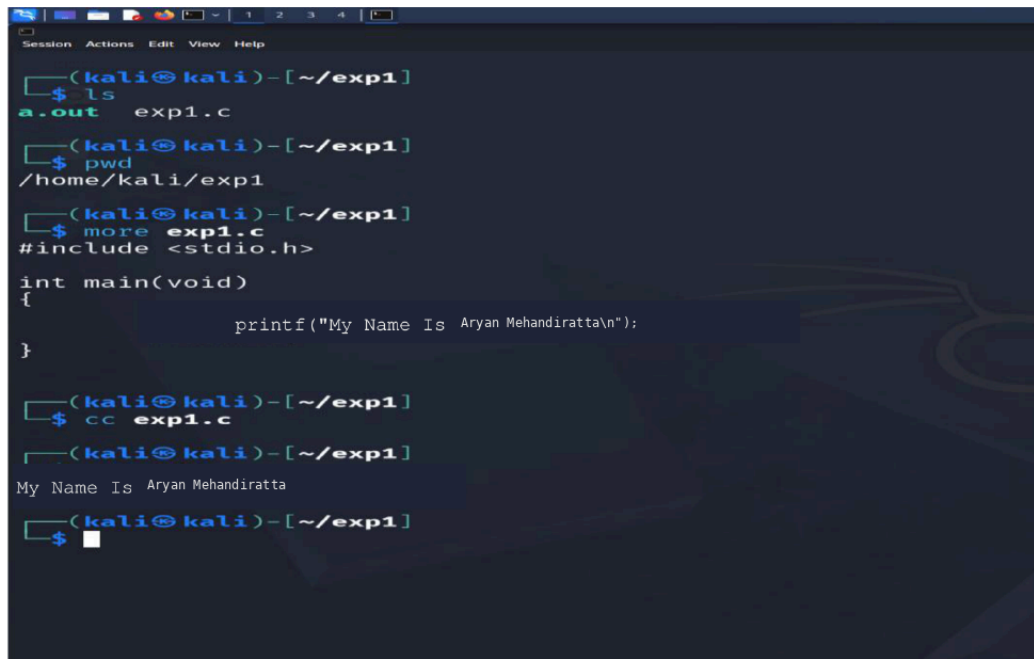
To display the contents of the source file, use:- `more exp1.c`

Compile the C program using the `cc` compiler:- `cc exp1.c`

After successful compilation, the compiler generates an executable file named `a.out`.

Execute the program using:- `./a.out`

Output:- My Name Is Aryan Mehendiratta



The screenshot shows a terminal window with a dark background and light-colored text. The prompt is `(kali@kali)~[/exp1]`. The user enters `ls`, and the output is `a.out exp1.c`. Then, the user enters `pwd`, and the output is `/home/kali/exp1`. Next, the user enters `more exp1.c`, and the terminal displays the contents of the file: `#include <stdio.h>`, `int main(void)`, `{`, `printf("My Name Is Aryan Mehendiratta\n");`, `}`. After that, the user enters `cc exp1.c`, and the prompt returns. Finally, the user enters `./a.out`, and the output is `My Name Is Aryan Mehendiratta`.

C) Basic Linux Commands

Command	Purpose	Example
pwd	Displays the present working directory.	pwd
mkdir	Creates a new directory.	mkdir test
rmdir	Removes an empty directory.	rmdir test
dir	Lists directory contents.	dir
ls	Lists files and directories.	ls
cd	Changes the current directory.	cd Documents
cd ..	Moves to the parent directory.	cd ..
clear	Clears the terminal screen.	clear
history	Shows previously executed commands.	history
man	Displays the manual page for a command.	man ls
whoami	Displays the current user name.	whoami
vi	Opens the vi text editor.	vi file.c
file	Identifies a file type.	file hello.c
rm	Removes a file.	rm file.txt
cp	Copies files or directories.	cp a.txt b.txt
mv	Moves or renames files.	mv old.txt new.txt
cat	Displays/combines file contents.	cat file.txt
more	Views text one screen at a time.	more file.txt
less	Views text interactively.	less file.txt
head	Displays the beginning of a file.	head file.txt
tail	Displays the end of a file.	tail file.txt
chmod	Changes file permissions.	chmod +x hello
>	Redirects output and overwrites a file.	ls > output.txt
>>	Redirects output and appends to a file.	echo Hi >> output.txt
<	Uses a file as standard input.	command < input.txt

Terminal Basic Commands

```
(kali㉿kali)-[~/exp1]
$ ls
a.out  exp1.c

(kali㉿kali)-[~/exp1]
$ pwd
/home/kali/exp1

(kali㉿kali)-[~/exp1]
$ mkdir test

(kali㉿kali)-[~/exp1]
$ ls
a.out  exp1.c  test

(kali㉿kali)-[~/exp1]
$ cd test

(kali㉿kali)-[~/exp1/test]
$ pwd
/home/kali/exp1/test

(kali㉿kali)-[~/exp1/test]
$ cd ..

(kali㉿kali)-[~/exp1]
$ rmdir test

(kali㉿kali)-[~/exp1]
$ ls
a.out  exp1.c

(kali㉿kali)-[~/exp1]
$ dir
a.out  exp1.c

(kali㉿kali)-[~/exp1]
$ whoami
kali

(kali㉿kali)-[~/exp1]
$ history
 1  mkdir demo1
 2  dir
 3  cd demo1
 4  vi first.c
 5  more first.c
 6  m
 7  more first.c
 8  vi first.c
 9  more first.c
10  cc first.c
11  dir
12  ./a.out
13  vi first.c
14  cc first.c
15  ./a.out
16  ls
17  pwd
18  mkdir exp1
19  pwd
20  cd
21  pwd
22  cd exp1
23  pwd
24  clear
25  ls
26  pwd
27  vi exp1.c
28  ls
29  cc exp1.c
30  ls
31  ./a.out
32  clear
33  ls
34  pwd
35  more exp1.c
36  cc exp1.c
37  ./a.out
38  clear
39  ls
40  clear
41  ls
42  pwd
43  mkdir test
44  ls
45  cd test
46  pwd
47  cd ..
48  rmdir test
49  ls
50  dir
51  whoami

(kali㉿kali)-[~/exp1]
```


File Commands

```
Session Actions Edit View Help

(kali@kali)-[~/exp1]
$ vi exp1.c

(kali@kali)-[~/exp1]
$ file exp1.c
exp1.c: C source, ASCII text

(kali@kali)-[~/exp1]
$ cp exp1.c copy.c

(kali@kali)-[~/exp1]
$ ls
a.out  copy.c  exp1.c

(kali@kali)-[~/exp1]
$ mv copy.c sample.c

(kali@kali)-[~/exp1]
$ ls
a.out  exp1.c  sample.c

(kali@kali)-[~/exp1]
$ cat exp1.c
#include <stdio.h>

int main(void)
{
    printf("My Name Is Aryan Mehendiratta\n");
    return 0;
}

(kali@kali)-[~/exp1]
$ more exp1.c
#include <stdio.h>

int main(void)
{
    printf("My Name Is Aryan Mehendiratta\n");
    return 0;
}
```

```
Session Actions Edit View Help

(kali@kali)-[~/exp1]
$ less exp1.c

(kali@kali)-[~/exp1]
$ head exp1.c
#include <stdio.h>

int main(void)
{
    printf("My Name Is Aryan Mehendiratta\n");
    return 0;
}

(kali@kali)-[~/exp1]
$ tail exp1.c
#include <stdio.h>

int main(void)
{
    printf("My Name Is Aryan Mehendiratta\n");
    return 0;
}

(kali@kali)-[~/exp1]
$ chmod +x sample.c

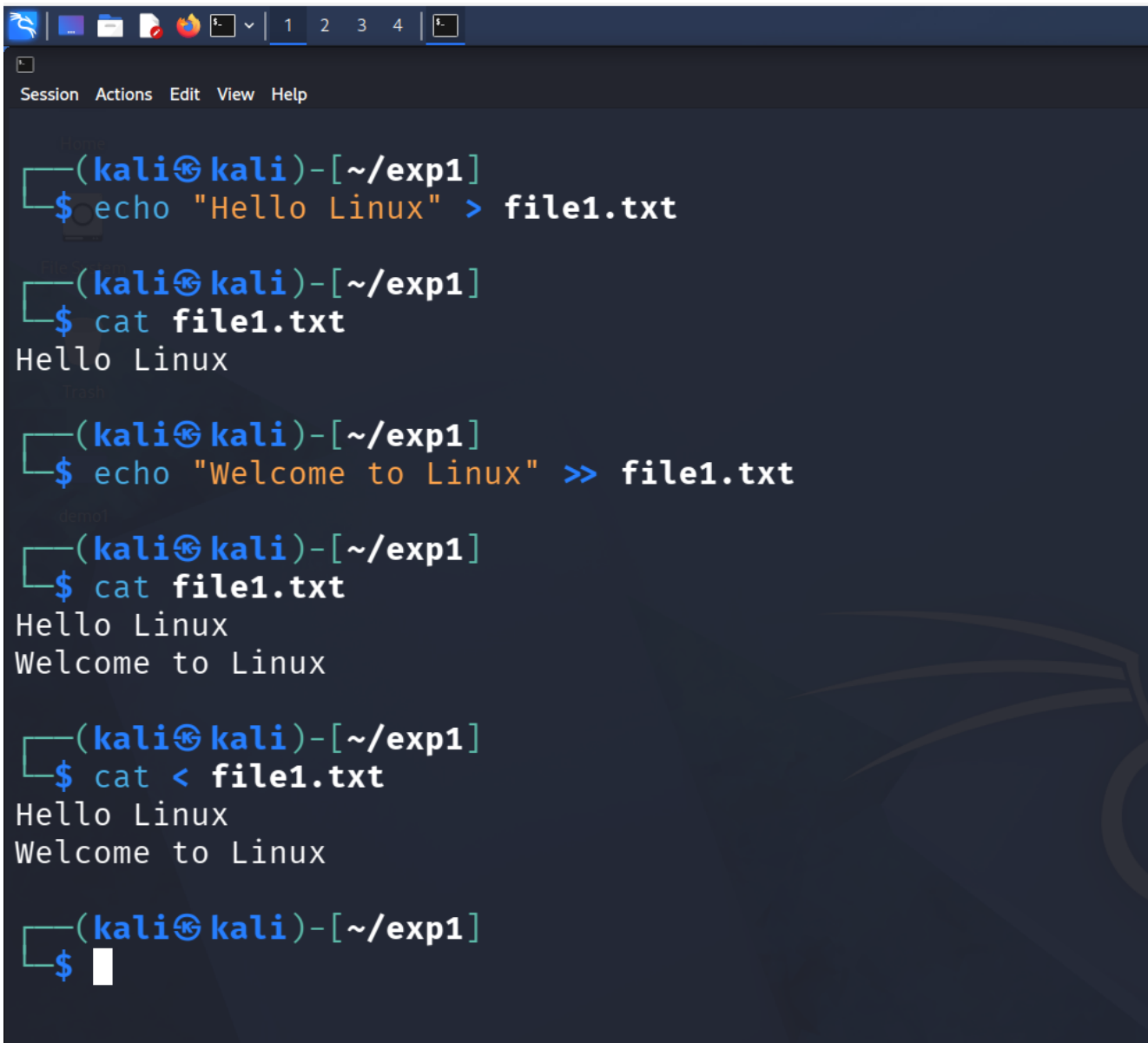
(kali@kali)-[~/exp1]
$ ls -l sample.c
sample.c

(kali@kali)-[~/exp1]
$ rm sample.c

(kali@kali)-[~/exp1]
$ ls
a.out  exp1.c

(kali@kali)-[~/exp1]
$
```


Redirection Commands

A terminal window titled "Redirection Commands" with a dark blue background. The window has a menu bar with "Session", "Actions", "Edit", "View", and "Help". The terminal shows a series of commands and their outputs. The prompt is "(kali@kali)-[~/exp1]". The commands are: 1. "\$ echo 'Hello Linux' > file1.txt" 2. "\$ cat file1.txt" followed by the output "Hello Linux". 3. "\$ echo 'Welcome to Linux' >> file1.txt" 4. "\$ cat file1.txt" followed by the output "Hello Linux" and "Welcome to Linux". 5. "\$ cat < file1.txt" followed by the output "Hello Linux" and "Welcome to Linux". The terminal ends with a prompt "\$" and a cursor.

```
(kali@kali)-[~/exp1]
$ echo "Hello Linux" > file1.txt

(kali@kali)-[~/exp1]
$ cat file1.txt
Hello Linux

(kali@kali)-[~/exp1]
$ echo "Welcome to Linux" >> file1.txt

(kali@kali)-[~/exp1]
$ cat file1.txt
Hello Linux
Welcome to Linux

(kali@kali)-[~/exp1]
$ cat < file1.txt
Hello Linux
Welcome to Linux

(kali@kali)-[~/exp1]
$
```

The Linux programming was set up successfully. A simple c program was created, compile using cc, and executed successfully. Basic Linux commands were also shown.